A RISCV 32 bits Forth in 454 bytes

Alvaro G. S. Barcellos,
may 2026
github.com/agsb/

To master riding bicycles you must ride bicycles

Proposal

A minimal functional Forth (engine and words)

Tuned for size (not for performance)

Expands the dictionary as any Forth

Fine tune Read, Eval, Print, Loop

Use of Minimal Indirect Thread Code

Current result https://github.com/agsb/milliForth-RiscV

454 bytes (without ELF headers) using:

88 bytes with headers, (11 Forth minimal words)

66 bytes with native syscalls, (linux)

300 bytes of native code (warp and beat)

no numbers, no names, no terminal input buffer,
no multitask, no multiuser, no rush.

why do count without ELF headers ?
because barebones don’t uses.

System details

riscv64-linux-gnu- (*gcc, *as, *ld, *objdump, *objcopy, *readelf)

-nostartfiles -nodefaultlibs -static -Oz -march=rv32gc -mabi=ilp32

"G" is simply a shorthand for IMAFD (General Purpose)

”C” is Standard Extension for Compressed Instructions

qemu-riscv32

-display none -bios none -serial mon:stdio -smp 1 -machine virt

Could be any barebone system with _init, _getc, _putc, _exit functions

Know to tune RISCV native code

A RISCV have 32 registers of 32 bits, X0 to X31, and 32 bits opcodes.

X0 hardwired to 0x0, X1 return address (link and jump), X2 stack pointer,

Intensive use Standard Compressed Instruction Set Extension, (3 : 1)

Use X8-X15 to make 16 bits opcodes, others make 32 bits opcodes,

No advantage in use SP as Forth stack, no magic automatic push or pull,

Use a register as constant memory pointer, and use relative offsets,

Use JAL and JALR with return address register,

Move values between registers, instead use of memory.

Look Ma, no terminal input buffer !

Chuck executes or compiles each word individually rather than line by line. In fact Chuck doesn't really have lines. [Jeff Fox, 2000]

No Terminal Input Buffer, only a input stream parser

A Forth engine is not an editor, no need of copy, paste, delete, insert, etc

Simply parse the tokens, valid ASCII characters between spaces,

and convert tokens into hashes.

How to be without names

WE STI.. DON.. SEE THE NEE. FOR 31 CHA..... NAM.. IN THE GEN.... CAS.

[Chuck Moore, 1983] we still don't see the need for 31 characters names in the general case

Do only hashes, no names, on linked list headers of dictionary

Link + *Hash*, no name length limit, small and fast,

Use 32 bits DJB2 hash, public domain, non cryptographic,

(KEY R> DUP DUP + DUP + DUP + DUP + DUP + + XOR >R)

In all words of Forth 2012 annex H, only F~ and D< have same djb2 hash

Forth IMMEDIATE flag is at bit 31 of the hash value.

Minimal world of words

Which could be the minimum set of words? It's a recurring theme.

My choices :

1. Words that can not be composed by another words

   NAND PLUS FETCH STORE NOTZEROIF

   NOTZEROIF return TRUE if TOS is not zero

2. Words that must exists to compile new words

   KEY EMIT COLON SEMIS EXIT USERAT

   USERAT return the address of user area variables

Warp and Beat ~ 256 bytes

Engines, mostly linear native code or two level calls

```
main cold warm miss abort quit                    \ setup
warp ( token find eval compile execute )          \ R.E.P.L.
token ( skip hash scan mask )                     \ parser
beat ( unnest next pick jump nest move )          \ M.I.T.C.
comma _init _getc _putc _exit                     \  support
```

M.I.T.C uses about 4 cycles for primitive and 14 cycles for compiled

Minimal vocabulary ~198 bytes

0#	TOS = zero if FALSE else TRUE then	U@	pointer to SP RP LATEST DP STATE
@	fetch, load a word from memory	!	store, store a word into memory
+	plus, add two values	NAND	NOT AND two values
:	colon, change to compile mode, etc	;	semis, change to execute mode, etc
KEY	read a byte from system stream	EMIT	write a byte into system stream
EXIT	ends a word and go to next word		

~ average of (8 + 10) bytes per word

Extra vocabulary ~ 236 bytes

This extra set can be selected during native code compilation.

NAN Not a Number, 0x80000000 to TOS	;$ DOCODE jump and link to ROS
LSHIFT left shift 1 to 31 bits	RSHIFT right shift 1 to 31 bits
ABORT restart the outer loop	BYE ends the Forth
. DOT output a hexadecimal number	$ DOLAR input a hexadecimal number
LIT place next cell from dictionary	

$ parse next token as 32-bit hexadecimal value, Eg. $ 4B1DCAFE

More 42 primitives ~ 678 bytes

This set can be selected during native code compilation.

DUP	DROP	OVER	SWAP	ROT	LIT
AND	OR	XOR	INVERSE	NEGATE	ALIGN
R>	>R	R@	-	M*	/MOD
=	<	U<	0>	C@	C!
TRUE	FALSE	CELL	BRANCH	0BRANCH	GBRANCH
STATE	LATEST	HEAP	CEIL	PEEK	PERFORM
S0	R0	SP@	RP@	SP!	RP!

those strange words

NAN	place 0x80000000 at TOS, like FALSE put [0] , TRUE put [-1]
GBRANCH	do branch if (TOS > 0), like 0BRANCH if (TOS = 0)
PERFORM	do ‘jump and link’ to address at TOS, to execute native code
;$	do ‘jump and link’ to address at IPT, to execute native code
$	next token is a signed hexadecimal 32-bit value
PEEK	a copy of HERE when start compilation, user variable
CEIL	a copy the address of last unused memory cell, user variable

How deep is your Forth

	average case			worst case		
primitive words	words per word	return depth	primitives per depth	word	return depth	primitives per depth
11 (454)	6	10	277	TO	16	1942
61 (1354)	7	4	18	TO	8	72

howdeep.awk over Forth files with one word per line, 221 words from ANSI core set.

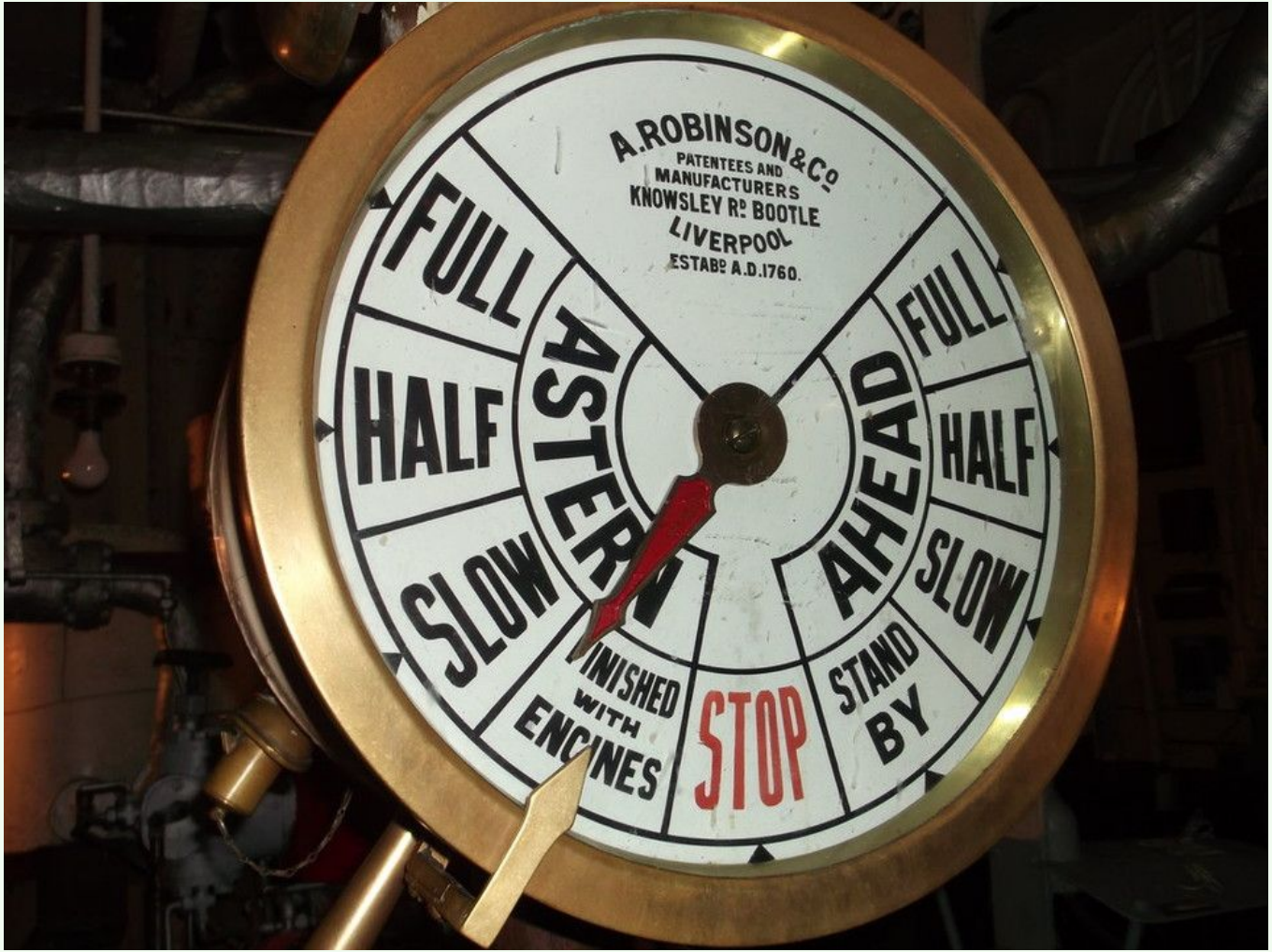
any word not defined is counted as primitive. Not optimized.

In average, with 61 words is like 16 times faster than with 11 words (27 in worst case)

What is need for

Forth internal variables in user area (U@)

SP	the pointer to user stack
RP	the pointer to return stack
HEAP	the address of next unused memory cell, aka DP
LAST	the address of last node of dictionary linked list, aka LATEST
STAT	the multi STATE of Forth, (execute, compile, [postpone, etc])
PEEK	the address of HEAP when start compiling, to be next LATEST
CEIL	the address of last unused memory cell, not initialized


A.ROBINSON & Co
PATENTEES AND
MANUFACTURERS
KNOWSLEY Rd BOOTLE
LIVERPOOL
ESTABd A.D.1760.
FULL
HALF
SLOW
ASTER
INISHED
WITH
ENGINES
STOP
STAND
BY
SLOW
HALF
FULL
AHEAD

some details

Only one flag, for immediate, 0x80000000. It also is NaN for numbers.

All BRANCHs are relative. Stacks of 36 cells deep.

The IP (s1) and USER (s0) pointers must be keep reserved.

A hash is 4 bytes, the average name length in Forth is 6 bytes.

Any word could start a compiled word. (no mandatory docol, dovar, etc)

Forth ANSI X3.215-1994, FALSE is 0x00000000, TRUE is 0xFFFFFFFF

Native multi STATE. (execute, compile, [postpone, more ?])

RAM_LIMIT = 0x40000 (64k words of 4 bytes, may be far more than this)

Work in Progress

Most of 454’s slowness is by lack of native code stack operators.

Full version, with 61 words, uses 1376 bytes, no inline code.

Compose (from various sources) the compiled word dictionary.

Test (really) on a Raspberry Pi RP 2350 RISC-V.

Create a version for Raspberry Pi RP 2040 ARM Cortex-M0.

Test (really) on a Raspberry Pi RP 2040 ARM Cortex-M0.

Create a version for 6502.

That’s all folks

Tuned for size not performance

PRIMITIVES:

FIGForth PDP-11 (16 bits, KA11/KD11, 1980) ~ 61 words, ? bytes
arrayForth (18 bits, F18B/GA144, 2010) ~ 32 opcodes/words
nybbleForth (16bits, verilog, 2017) ~ 12 words, ? , ?
jonesForth (16 bits, X86, 2007) ~ 75 words, ? bytes, ELF
sectorForth (16 bits, x86, 2020) ~ 17 words, 512 bytes, ELF
milliForth (16bits, x86, 2023) ~ 13 words, 336 bytes, ELF
Primitive Words To Bootstrap A Forth (2024) ~ 14 words
milliForth (8 bits, 6502) ~ 11 words, 584 bytes, barebone/cc65
milliForth (32 bits, RISCV) ~ 11 words, 454 bytes, barebone/ELF

Subroutines, without magic push or pull

load-save ISA

```
# load address to s0
aiupc s0,  0xNNNNN #; 20bits
lw s0,       0xMMM #; 12 bits

# load indirect
lw a0, 0 ( s0 )
lw a1, 4 ( s0 )

# save indirect
sw a0, 0 ( s0 )
sw a1, 4 ( s0 )
```

```
# do whatever

# save last ra, push
addi sp, sp, -4
sw ra, 0 (sp)
# call the token in a0
jalr ra, 0 (a0)
# load last ra, pull
lw ra, 0 (sp)
addi sp, sp, 4

# do whatever
```

```
# at address in a0
# save last ra, push
addi sp, sp, -4
sw ra, 0 (sp)

# do whatever

# load last ra, pull
lw ra, 0 (sp)
addi sp, sp, 4
# return to address in ra
 jalr 0, 0 (ra)
```

Jump and Link (32 bits)

Jump to subroutine

; push RA, one deep

SP ← SP − 4

[SP] ← RA

; link and jump

RA ← PC + 4

PC ← EA

Return from subroutine

; pull RA, one deep

RA ← [SP]

SP ← SP + 4

; link and jump

ZR ← PC

PC ← RA

PC, program counter; EA, effective code address; the registers defaults to ZR (x0), wired zero; RA (x1), return address; SP (x2), stack pointer;

Laziness and Complexity

Why still not KEY? in native code ? 2026 ?

6502, 37 pages, https://www.princeton.edu/~mae412/HANDOUTS/Datasheets/6502.pdf

BIOS, Basic Input Output System, 96 pages,
https://www.dmtf.org/sites/default/files/standards/documents/DSP0134_2.6.1.pdf

RISCV, the RISC-V unprivileged architecture, 696 pages,
https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf,

ACPI, Advanced Configuration and Power Interface Specification, 1202 pages,
https://uefi.org/sites/default/files/resources/ACPI_Spec_6.6.pdf

UEFI, Unified Extensible Firmware Interface, 2301 pages,
https://uefi.org/sites/default/files/resources/UEFI_Spec_Final_2.11.pdf

no numbers and no rush.

```
: -1 U@ 0# ;
 : 0 -1 -1 NAND ;

 : TRUE -1 ;
 : FALSE 0 ;

 : 1 -1 -1 + -1 NAND ;
 : 2 1 1 + ;
 : 4 2 2 + ;
 : CELL 4 ;

 : SP U@ ;
 : RP SP CELL + ;

 : SP@ SP @ CELL + ;
 : RP@ RP @ CELL + ;
```

```
 : DUP SP@ @ ;
 : OVER SP@ CELL + @ ;

 : NOT DUP NAND ;
 : AND NAND NOT ;

 : BRANCH  RP@ @ DUP @ + RP@ ! ;
 : 0BRANCH 0# NOT RP@ @ @ CELL -  AND
                RP@ @ + CELL + RP@ ! ;

: SWAP OVER OVER
       SP@ CELL + CELL + CELL + !
       SP@ CELL + ! ;

: OR NOT SWAP NOT AND NOT ;
: NOR OR NOT ;
```

```
pull

LW a4, SPTR (s0);

LW a3, 0 (a4);

ADDI a4, a4, 4;

SW a4, SPTR (s0);

push

LW a4, SPTR (s0);

ADDI a4, a4, -4;

SW a3, 0 (a4);

SW a4, SPTR (s0);
```

Myself

I ‘m Geologist, Master in Geophysics,

I use computers for data acquisition and calculations,

and an electronics and circuits hobbyist.

I discovered Forth in 1980, in Byte Magazine, in the UFRJ-BR library,

and rediscovered Forth in 2019, with the Forth2020 forum.

My projects at https://github.com/agsb

References

https://groups.google.com/g/comp.lang.forth/c/NS2icrCj1jQ

https://archive.org/details/the-road-towards-a-minimal-forth-architecture

https://www.forth.org/fig-forth/fig-forth_PDP-11.pdf

https://rwmj.wordpress.com/2010/08/07/jonesforth-git-repository/

https://github.com/cesarblum/sectorforth

https://github.com/fuzzballcat/milliForth

https://github.com/agsb/milliForth-6502

https://github.com/agsb/milliForth-RiscV

References

https://en.wikipedia.org/wiki/RISC-V

https://theforth.net/package/minimal

https://github.com/larsbrinkhoff/nybbleForth

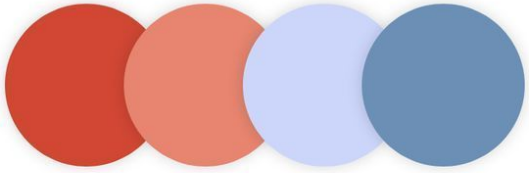
https://github.com/meithecatte/miniforth/tree/post1

https://github.com/larsbrinkhoff/nybbleForth

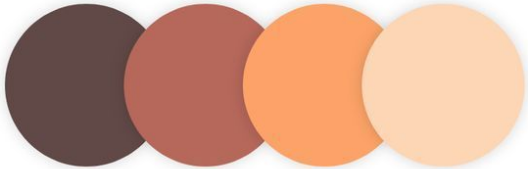
https://www.youtube.com/watch?v=Ok-1E8xwo90

https://www.greenarraychips.com/home/documents/greg/DB001-171107-F18A.pdf

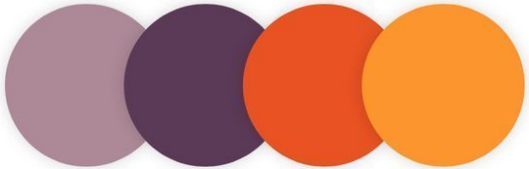

STOP
DEAD SLOW
SLOW
HALF
ASTERN
DEAD SLOW
SLOW
AHEAD
HALF



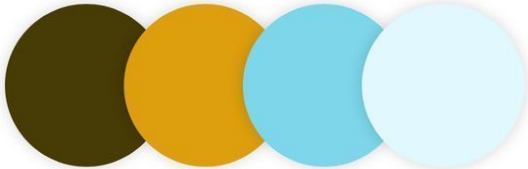
#D14734 #E78570 #CCD6FB #6C8FB5



#614947 #B6685B #FDA369 #FDD6B6



#AD8997 #5B3A57 #E95223 #FC952D

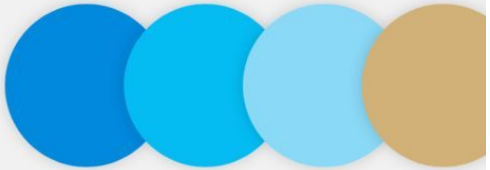


#473B06 #DD9F0D #7DD6EA #E1F8FF

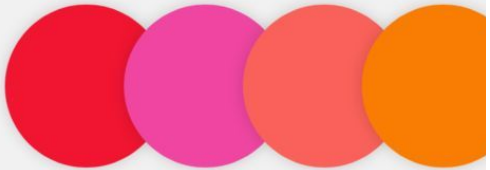
Made with dopely.top @dopelyapp

#E68F59 #073E6B #1973AE #A1D8EC

#333C31 #97B45D #A49752 #D7B56B

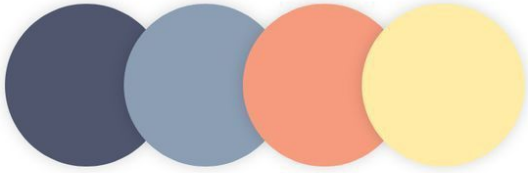


#0189DD #04BCF2 #8AD9F7 #D2B178

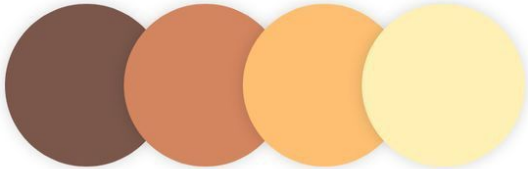


#F11532 #F046A1 #FA615B #F97D03

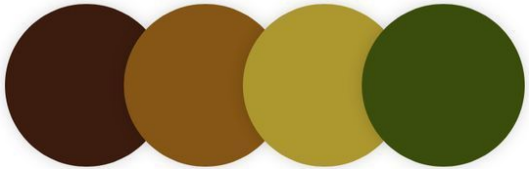
Made with dopely.top @dopelyapp



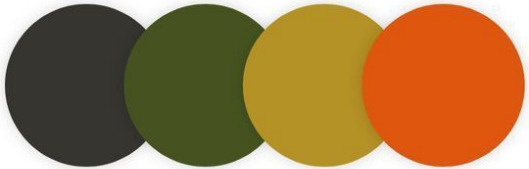
#4F566E #8C9EB4 #F79B7E #FFECA7



#7B574B #D3855F #FEBF72 #FFF0B6



#3C1C0F #855615 #AD972F #3B4D0C



#36352F #465321 #B59226 #DF560E

Made with dopely.top @dopelyapp